

6. OpenClaw Gateway Operation Manual

Gateway is OpenClaw's persistent process: it maintains message channel connections, hosts the control and event plane, and serves as the unified entry point for sessions, routing, and channel state. Simply put: whether you can "connect to OpenClaw", whether the console can open, and whether messages can come in often depends on whether the Gateway is running properly.

6. OpenClaw Gateway Operation Manual

1. When Do You Need to Care About the Gateway
2. Fastest Way to Start (Local)
3. What "External Interfaces" Does the Gateway Provide
4. Hot Reload and Changes That "Need Restart"
5. Authentication and Remote Access (Easiest Pitfalls)
 - 5.1 Gateway Authentication (token/password)
 - 5.2 Recommended Way for Remote Access: Tunnel First, Then Access
 - 5.3 Console Won't Open via Pure HTTP (device identity required)
6. Multi-Instance and "Rescue Bot" Mode
7. Troubleshooting Sequence, Usually Can Locate
 - 7.1 Check Status First
 - 7.2 Gateway Won't Start: Configuration Validation Failed
 - 7.3 Common Phenomenon: Service Installed But Not Actually Running

1. When Do You Need to Care About the Gateway

Most of the time, you just need to let it run stably; you only need to specifically check/learn about the gateway in these scenarios:

- Opening the console on the RDK browser often gives errors, planning to switch to TUI or remote access
- Configuration changed but "feels like it didn't take effect", suspect hot reload/restart wasn't triggered
- Want to open the console to LAN or access via SSH/Tailscale remotely
- Want multi-instance isolation (e.g., "rescue bot"/redundancy)
- Port conflicts, process won't start, service "looks installed but not running"

2. Fastest Way to Start (Local)

Execute on the gateway host (e.g., RDK):

```
openclaw gateway
```

Common parameters:

```
# Print more complete debug/trace info to current terminal (helpful for troubleshooting)
openclaw gateway --port 18789 --verbose

# When port is occupied, try to kill the listener occupying the port and force start
openclaw gateway --port 18789 --force
```

Port priority (high to low): `--port` > `OPENCLAW_GATEWAY_PORT` > `gateway.port` > default `18789`.

3. What "External Interfaces" Does the Gateway Provide

The same port (default 18789) provides both WebSocket control plane and HTTP service (control interface, hooks, A2UI, etc.) at once. This is "single-port multiplexing".

Common points you will use:

- Local console: `http://127.0.0.1:18789/`
- OpenAI Chat Completions compatible interface: `/v1/chat/completions`
- OpenResponses interface: `/v1/responses`
- Tool invocation interface: `/tools/invoke`

Additionally, the gateway starts Canvas static file service by default (default port 18793), providing editable HTML/A2UI resources (served from `~/.openclaw/workspace/canvas` by default). To disable, set `canvasHost.enabled=false` or `OPENCLAW_SKIP_CANVAS_HOST=1`.

4. Hot Reload and Changes That "Need Restart"

The gateway monitors `~/.openclaw/openclaw.json` (or the path specified by `OPENCLAW_CONFIG_PATH`). Configuration updates **are usually applied automatically**. The default reload mode is `gateway.reload.mode="hybrid"`: safe changes are hot-applied, critical changes trigger restart.

You can explicitly configure reload behavior:

```
{
  "gateway": {
    "reload": { "mode": "hybrid", "debounceMs": 300 }
  }
}
```

Experience suggestions:

- Changing models, agents, routing, and other business configurations usually don't require manual restart
- Changing `gateway.*` (port, binding, authentication, TLS, HTTP, etc.) are infrastructure changes and more likely to trigger restart

5. Authentication and Remote Access (Easiest Pitfalls)

5.1 Gateway Authentication (token/password)

Gateway has authentication enabled by default: you can set `gateway.auth.token` (or `OPENCLAW_GATEWAY_TOKEN`) or `gateway.auth.password`. Clients need to carry `connect.params.auth.token/password` when connecting.

If you use the wizard flow, a token is usually generated by default (even if only bound to loopback).

5.2 Recommended Way for Remote Access: Tunnel First, Then Access

Most recommended: Tailscale/VPN. Next: SSH tunnel.

Example (map remote 18789 to local 18789):

```
ssh -N -L 18789:127.0.0.1:18789 user@host
```

Then you access locally:

- Web: `http://127.0.0.1:18789/`
- WS: `ws://127.0.0.1:18789`

Note: Even through the tunnel, if the gateway is configured with a token, clients still need to bring the token to connect.

5.3 Console Won't Open via Pure HTTP (device identity required)

If you open the console at `http://<lan-ip>:18789/` on LAN, the browser may be in a non-secure context, causing WebCrypto to be restricted, thus unable to generate device identity, resulting in `device identity required / connect failed`.^[^gateway_troubleshooting]

Priority fix route:

- Open locally: `http://127.0.0.1:18789/`
- For remote scenarios, use Tailscale Serve to provide HTTPS
- If HTTP must be used, enable `gateway.controlui.allowInsecureAuth: true` and use the gateway token (token mode only, not device identity/pairing)^[^gateway_troubleshooting]

6. Multi-Instance and "Rescue Bot" Mode

Only try this if needed.

Usually one host running one gateway is sufficient; only when redundancy or strong isolation is needed (e.g., rescue bot) should multiple gateway instances be considered.

The core principle for multi-instance is "complete isolation + no conflicts":

- Different `gateway.port`
- Different `OPENCLAW_CONFIG_PATH`
- Different `OPENCLAW_STATE_DIR`
- Different workspaces (`agents.defaults.workspace`)

In the dev config file, you can quickly start a fully isolated dev instance without affecting the main environment:

```
openclaw --dev setup
openclaw --dev gateway --allow-unconfigured
openclaw --dev status
openclaw --dev health
```

7. Troubleshooting Sequence, Usually Can Locate

7.1 Check Status First

Execute in order:[^gateway_troubleshooting]

```
openclaw status
openclaw status --all
openclaw status --deep
```

Common supplementary commands:

```
openclaw gateway probe
openclaw channels status --probe
openclaw gateway status
openclaw logs --follow
openclaw doctor
```

7.2 Gateway Won't Start: Configuration Validation Failed

OpenClaw configuration is strict schema validation: unknown keys, type errors, or invalid values may cause the gateway to refuse to start.

When validation fails, usually only diagnostic commands are available (e.g., `openclaw doctor` / `logs` / `health` / `status`). Recommended to directly:

```
openclaw doctor
```

7.3 Common Phenomenon: Service Installed But Not Actually Running

If you use systemd/launchd/schtasks to install the gateway as a service, showing "loaded/installed" doesn't equal the process running. Priority check:[^gateway_troubleshooting]

```
openclaw gateway status
openclaw logs --follow
```